Validation of Reduced Fluorescence Rendering

Technical companion to the `spectral-reduction-validation` repository

Mohammed Douidy
Master's internship, supervised by Pascal Barla & Morgane Gerardin

September 2026

Abstract

This document describes what the five scripts of the repository compute and in which colour space each quantity lives. It assumes familiarity with the two source papers (Fichet et al. 2024; Belcour et al. 2025). The theory as implemented is in Sections 2 to 5. The conventions of the Godot port and the checks that tie it to the Python code are in Section 7. Section 8 collects the choices that could be mistaken for bugs.

Contents

1 Scope and reading guide

The goal of the internship's validation phase was to answer one question with numbers before writing any shader code: *how much colour accuracy is lost when the per-wavelength transport of light through a fluorescent material is replaced by a small $K \times K$ matrix, over one and over many bounces?*

The repository answers it in three steps, each a script with committed output:

1. `validate_reflectance.py` – reflectance only, $K = 3$ (XYZ). Settles how an authored colour becomes a physically valid spectrum, and whether a *hybrid* transport (diagonal first bounce, reduced afterwards) is sound.
2. `validate_fluorescence.py` – reflectance plus fluorescence, $K = 4$ (XYZU). Experiments with the closed-form Gaussian fluorescence model of Belcour et al. against a full spectral reference.
3. `generate_godot_matrices.py`, `verify_fbar_convention.py`, `verify_godot_shader.py` – Emit every constant the shader hardcodes, and prove the shader's algebra reproduces the validated pipeline to machine precision.

A sixth script, `fig_upscale_diag.py`, is a standalone reproduction of a supplementary figure by the paper's authors, kept because it was the reference against which our own multi-bounce behaviour was aligned.

Throughout, **spectra are column vectors on a 1 nm grid**, matrices act on the left, and "reduced" always means projected onto a colour-matching-function basis by the dual-basis construction of Section 2.

2 Spectral reduction

2.1 Basis, dual basis, coordinates

Let $\mathbf{S} \in \mathbb{R}^{N \times K}$ hold K colour-matching functions (CMFs) sampled at N wavelengths, one per column. A spectrum $\mathbf{s} \in \mathbb{R}^N$ has colour coordinates

$$\mathbf{c} = \mathbf{S}^\mathsf{T}\mathbf{s} \in \mathbb{R}^K. \tag{1}$$

Going back is ill-posed; the construction of Fichet et al. (2024) uses the *dual basis*

$$\tilde{\mathbf{S}} = \mathbf{S}\,(\mathbf{S}^\mathsf{T}\mathbf{S})^{-1}, \qquad \mathbf{S}^\mathsf{T}\tilde{\mathbf{S}} = \mathbf{I}_K, \tag{2}$$

so that $\hat{\mathbf{s}} = \tilde{\mathbf{S}}\mathbf{c}$ is the unique spectrum in the span of $\tilde{\mathbf{S}}$ with the prescribed coordinates: $\mathbf{S}^\mathsf{T}\hat{\mathbf{s}} = \mathbf{c}$ exactly. Every script builds $\tilde{\mathbf{S}}$ as `S @ pinv(S.T @ S)` and prints $\max|\mathbf{S}^\mathsf{T}\tilde{\mathbf{S}} - \mathbf{I}|$ as a self-check (observed 10^{-15}).

2.2 Reducing a linear spectral operator

A linear light–matter interaction is an operator $\mathbf{P} \in \mathbb{R}^{N\times N}$ mapping an incident spectrum to an outgoing one. Its reduction is

$$\bar{\mathbf{P}} = \mathbf{S}^\mathsf{T}\,\mathbf{P}\,\tilde{\mathbf{S}} \in \mathbb{R}^{K\times K}, \tag{3}$$

which is exact whenever the incident spectrum lies in the span of $\tilde{\mathbf{S}}$: then $\mathbf{S}^\mathsf{T}\mathbf{P}\mathbf{s} = \mathbf{S}^\mathsf{T}\mathbf{P}\tilde{\mathbf{S}}\mathbf{c} = \bar{\mathbf{P}}\mathbf{c}$. For any other incident spectrum the error is the part of $\mathbf{s}$ that the basis cannot see, the *metameric black* $(\mathbf{I} - \tilde{\mathbf{S}}\mathbf{S}^\mathsf{T})\mathbf{s}$, transported by $\mathbf{P}$ back into the visible. The reduced pipelines below have no other source of error; the experiments measure its size.

The naive alternative, applying the colour coordinates component-wise ($\mathbf{c}_{\text{out}} = \boldsymbol{\rho} \circ \mathbf{c}_{\text{in}}$), is not a reduction of any spectral operator and conserves neither energy nor chromaticity across bounces. It appears in every figure as the "Naive" baseline.

2.3 Column normalisation

All scripts normalise the CMFs so that every column of $\mathbf{S}$ sums to one (`S = S_raw / S_raw.sum(axis=0)`). With this choice the coordinates of a flat reflectance $\mathbf{r} = 1$ are $\boldsymbol{\rho} = (1, \ldots, 1)$, so an albedo in $[0, 1]$ per channel is meaningful, and a perfect white under an illuminant normalised to $\mathbf{L}^\mathsf{T}\mathbf{S}_{:,Y} = 1$ has $Y = 1$. The conversion between these *normalised* XYZ coordinates and standard CIE XYZ is a diagonal scaling,

$$\text{std} \to \text{norm}: \ \text{diag}\big(\textstyle\sum \bar{y} \,/\, \sum \bar{x},\ 1,\ \sum \bar{y} / \sum \bar{z}\big), \tag{4}$$

and the sRGB matrices used for display are composed with it (`LIN_TO_NORM`, `NORM_TO_LIN`). Mixing normalised and un-normalised quantities was the source of an early "colours 107× too dark" bug; the current scripts keep every quantity in one convention and convert only at the display boundary.

2.4 The two bases used

Script	K	Channels	Grid	CMF source
`validate_reflectance.py`, `fig_upscale_diag.py`	3	X Y Z	380–780 nm (401)	CIE 2006 2°
`validate_fluorescence.py`, other scripts	4	X Y Z U	300–799 nm (500)	Fichet 2024 `xyzu.csv`

Table 1: Reflectance-only work uses the classical 3-channel basis. Fluorescence needs a fourth channel U (a UV Gaussian) so that absorption in the UV can be represented, which in turn requires the wider grid. The two are not interchangeable; see Section 8.

3 From an authored colour to a spectrum

Every material is authored as an sRGB triple. To build a spectral reference the scripts need a reflectance spectrum $\mathbf{r}(\lambda) \in [0, 1]$ consistent with it. This is under-determined (any metamer will do). The two scripts use two different constructions.

3.1 Dual-basis reconstruction with alpha scaling

The direct route is $\mathbf{r} = \tilde{\mathbf{S}}\,\mathbf{c}$ with $\mathbf{c}$ the XYZ of the colour. It is exact in coordinates but *unbounded*: for saturated colours $\max_\lambda \mathbf{r}(\lambda) > 1$, which is unphysical and, raised to the n-th power over n bounces, blows up. `validate_reflectance.py` therefore scales the colour first,

$$\alpha = \frac{0.9}{\max_\lambda(\tilde{\mathbf{S}}\mathbf{c})(\lambda)}, \qquad \mathbf{c}_{\text{eff}} = \alpha\,\mathbf{c} \text{ (applied in linear sRGB)}, \qquad \mathbf{r} = \text{clip}(\tilde{\mathbf{S}}\mathbf{c}_{\text{eff}}, 0, 1), \tag{5}$$

so that the reconstructed spectrum peaks at about 0.9. Because α is applied to the linear RGB before conversion, chromaticity is preserved and only brightness moves. The printed `spectrum XYZ diff` confirms the round trip $\mathbf{S}^\mathsf{T}\mathbf{r} = \mathbf{c}_{\text{eff}}$ to 10^{-16}.

3.2 Jakob–Hanika sigmoid upsampling

`validate_fluorescence.py` instead uses the model of Jakob & Hanika (2019): a spectrum is the sigmoid of a quadratic in wavelength,

$$x(\lambda) = c_0\lambda^2 + c_1\lambda + c_2, \qquad \mathbf{r}(\lambda) = \tfrac{1}{2} + \tfrac{1}{2}\,\frac{x}{\sqrt{1+x^2}}, \tag{6}$$

with (c_0, c_1, c_2) read from a precomputed table (`data/srgb.coeff`, trilinear interpolation in RGB). The sigmoid maps $\mathbb{R} \to (0, 1)$, so the result is **bounded by construction**: no scaling step is needed and saturated primaries, notably reds where the CMF span is poor, are reconstructed smoothly. The table is fitted on the visible range; evaluating it on 300–799 nm extrapolates the fitted polynomial below 380 nm, which is smooth but not measured (Section 8).

3.3 Which one, where, and why

Jakob–Hanika serves as the ground-truth albedo for the fluorescence experiment because it is valid without intervention. The dual basis is the reconstruction inside the reduced model (it is what $\tilde{\mathbf{S}}$ does in Eq. (3)), and the alpha-scaled variant of Section 3.1 is tested on its own by the reflectance script.

They are two competing pipelines and both are kept. Neither is currently applied to authored albedos by the engine: the shader converts the albedo texture straight from linear sRGB to normalised XYZ and takes ρ_U by the Method-2 projection, with no scaling step (Section 7.2). Alpha scaling is therefore a validated recommendation that the port has not yet taken up.

4 Reflectance transport

4.1 The reduced reflectance basis

A Lambertian reflectance is the diagonal operator $\mathbf{P} = \text{diag}(\mathbf{r})$. Writing the reconstruction $\mathbf{r} = \tilde{\mathbf{S}}\boldsymbol{\rho}$ and substituting in Eq. (3) gives a reduced operator that is *linear in the albedo coordinates*:

$$\bar{\mathbf{R}}(\boldsymbol{\rho}) = \mathbf{S}^\mathsf{T}\,\text{diag}(\tilde{\mathbf{S}}\boldsymbol{\rho})\,\tilde{\mathbf{S}} = \sum_{k=1}^{K} \rho_k\,\mathbf{R}_k, \qquad \mathbf{R}_k = \mathbf{S}^\mathsf{T}\,\text{diag}(\tilde{\mathbf{S}}_{:,k})\,\tilde{\mathbf{S}}. \tag{7}$$

The K constant matrices $\mathbf{R}_k$ depend only on the basis (`Rk = S.T @ diag(Sb[:,k]) @ Sb`); at run time a material costs one weighted sum. The Godot shader stores them as `Rx, Ry, Rz, Ru`.

4.2 Three transport schemes

With $\mathbf{c}_0 = \mathbf{S}^\mathsf{T}\mathbf{L}_0$ the reduced illuminant and $\mathbf{D} = \mathrm{diag}(\boldsymbol{\rho})$, bounce $n \geq 1$ is evaluated as

$$\text{Reduced:} \quad \mathbf{c}_n = \bar{\mathbf{R}}^{\,n}\,\mathbf{c}_0, \tag{8}$$

$$\text{Hybrid:} \quad \mathbf{c}_n = \bar{\mathbf{R}}^{\,n-1}\,\mathbf{D}\,\mathbf{c}_0, \tag{9}$$

$$\text{Naive:} \quad \mathbf{c}_n = \boldsymbol{\rho}^{\circ n} \circ \mathbf{c}_0, \tag{10}$$

against the reference $\mathbf{c}_n^{\mathrm{ref}} = \mathbf{S}^\mathsf{T}(\mathbf{r}^{\circ n} \circ \mathbf{L}_0)$ computed per wavelength. The Hybrid scheme applies the component-wise product only for the directly lit bounce and the reduced matrix for every later one. It was motivated by the observation that under a broadband illuminant the first bounce of Naive is already exact in coordinates, while from the second bounce on the reduced matrix tracks the reference far better; the figures test whether that holds across illuminants.

4.3 Illuminant handling

Illuminants are CIE standard SPDs, normalised to $Y = 1$ for a perfect white ($\mathbf{L}^\mathsf{T}\mathbf{S}_{:,Y} = 1$) so that the six rows of each figure differ in chromaticity only. The reflectance script additionally *smooths* the illuminant before use,

$$\mathbf{L}_{\mathrm{smooth}} = \tilde{\mathbf{S}}\,(\mathbf{S}^\mathsf{T}\mathbf{L}), \text{ re-normalised to } Y = 1, \tag{11}$$

and feeds $\mathbf{L}_{\mathrm{smooth}}$ to the spectral reference as well. Without this, a spiky illuminant such as HP1 would penalise the reduced schemes for spectral detail that no K-dimensional model can represent: the comparison would measure the basis's expressiveness, not the transport's accuracy. With it, reference and models see the same reduced light and the residual error is the transport error alone.

4.4 Experiment and figure

Eight sRGB materials × six illuminants (D65, D50, HP1, E, A, LED-B1) × 20 bounces. Figure 1 shows one material. In each swatch column the top half is the spectral reference and the bottom half the model, one band per bounce; the ΔE_{2000} panel plots the three schemes against the reference; the last two columns show the smoothed illuminant and the reconstructed $\mathbf{r}(\lambda)$, so the figure carries its own inputs.

5 Fluorescence

5.1 Spectral model

Fluorescence moves energy *between* wavelengths: light absorbed at λ_i is re-emitted at $\lambda_o > \lambda_i$. The reference builds the reradiation matrix explicitly,

$$\mathbf{F}[\lambda_o, \lambda_i] = \bar{\alpha}\, a_{\max}\, e^{-\frac{(\lambda_i-\mu_a)^2}{2\sigma_a^2}}\, e^{-\frac{(\lambda_o-\mu_e)^2}{2\sigma_e^2}}\, \mathbb{1}[\lambda_o > \lambda_i], \qquad a_{\max} = \Big(\max_{\lambda_i} \sum_{\lambda_o} \mathbf{F}[\lambda_o, \lambda_i]\Big)^{-1}, \tag{12}$$

a separable absorption–emission Gaussian pair masked by the Stokes-shift Heaviside, normalised so that the most absorbing wavelength re-emits at most its full energy ($\bar{\alpha} \in [0, 1]$ then scales the yield). One complete interaction is

$$\mathbf{P} = \mathrm{diag}(\mathbf{r}) + \mathbf{F}\ \mathrm{diag}(\mathbf{1} - \mathbf{r}), \tag{13}$$

which reads: the fraction $\mathbf{r}$ is reflected at the same wavelength (diagonal), the fraction $1 - \mathbf{r}$ is absorbed and a part of it re-emitted elsewhere (off-diagonal). This is the paper's $\mathbf{P} = \mathbf{R} + \mathbf{F}(\mathbf{I} - \mathbf{R})$ at full spectral resolution, 500×500 here.

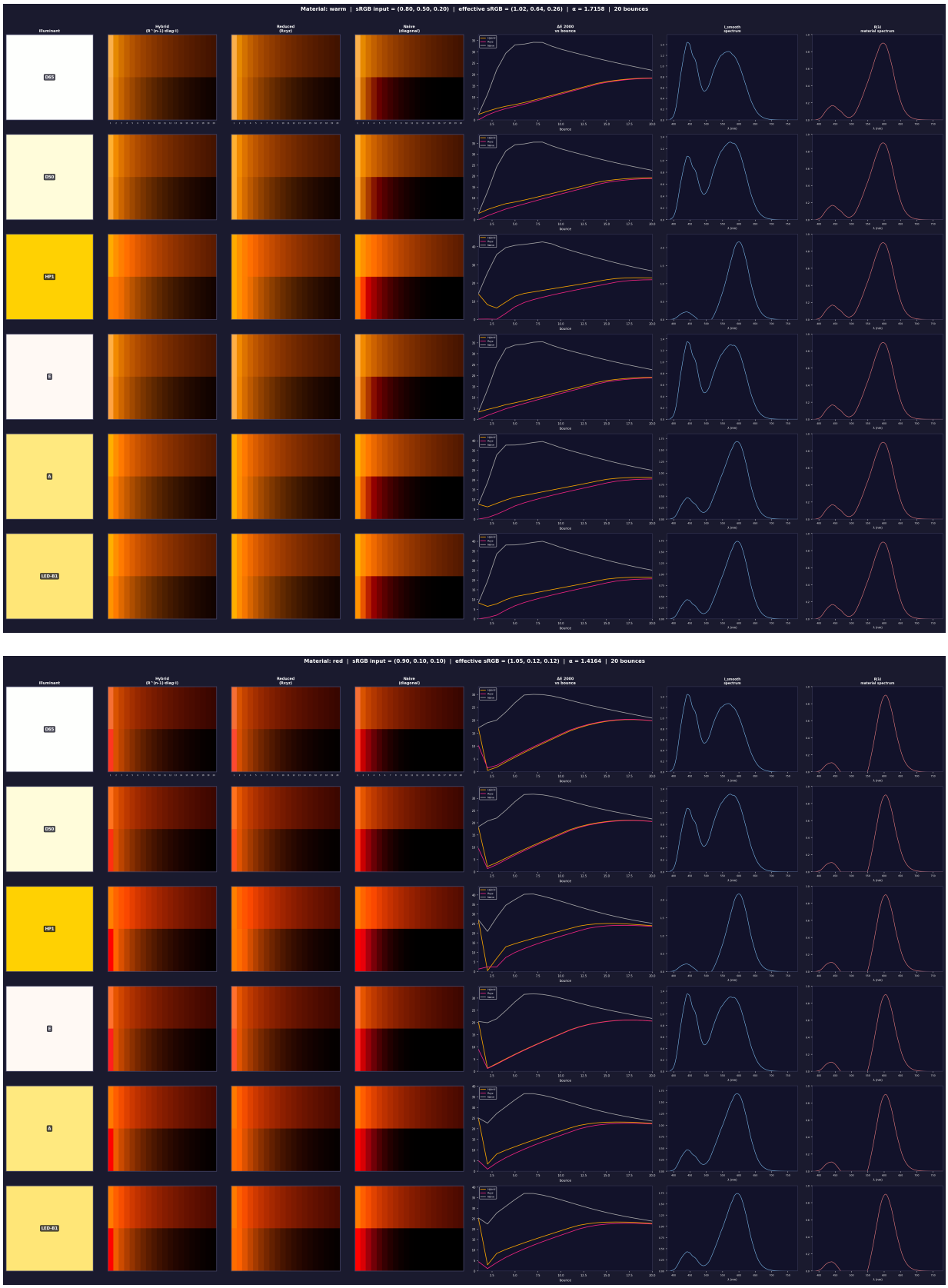

Figure 1: `validate_reflectance.py`, materials *warm* (top) and *red* (bottom). Columns: illuminant, Hybrid, Reduced, Naive, ΔE_{2000} vs bounce, $\mathbf{L}_{\text{smooth}}$, $\mathbf{r}(\lambda)$. The alpha scaling is visible in the title (effective sRGB and α). The saturated red shows the regime where the ΔE_{2000} clipping convention matters (Section 8).

5.2 Analytic Gaussian model (what the shader evaluates)

Reducing Eq. (13) gives $\bar{\mathbf{P}} = \bar{\mathbf{R}} + \bar{\mathbf{F}}(\mathbf{I} - \bar{\mathbf{R}})$ with $\bar{\mathbf{F}} = \mathbf{S}^\mathsf{T}\mathbf{F}\tilde{\mathbf{S}}$. The brute-force $\bar{\mathbf{F}}$ requires the 500×500 matrix; the contribution of Belcour et al. (2025) is a closed form that needs only the five Gaussian parameters. The CMFs are first approximated by $M = 5$ one-dimensional Gaussians (the paper's Table 1), $\mathbf{S} \approx \mathbf{G}\,\mathbf{T}_G$ with $\mathbf{G} \in \mathbb{R}^{N\times 5}$, and the correction $\mathbf{C} = \left((\mathbf{G}\mathbf{T}_G)^\mathsf{T}(\mathbf{G}\mathbf{T}_G)\right)^{-1}$ compensates for the fit. Products of Gaussians are Gaussians and their integral against the Heaviside is an error function, so each entry of $\mathbf{F}^\circ \in \mathbb{R}^{5\times 5}$ has a closed form (paper Eq. 28) and

$$\bar{\mathbf{F}}_{\text{an}} = \mathbf{T}_G^\mathsf{T}\,\mathbf{F}^\circ\,\mathbf{T}_G\,\mathbf{C}. \tag{14}$$

The scripts compute both $\bar{\mathbf{F}}_{\text{bf}}$ and $\bar{\mathbf{F}}_{\text{an}}$ and print their difference for every material. Two implementation points:

- $\mathbf{T}_G$ is obtained by a *dense least-squares fit* (`lstsq(G, S4)`), not by the paper's canonical 0/1 grouping (Eq. 29). The grouping assumes the standard-scale CMFs the Gaussians were fitted to; our basis is column-normalised, so the dense fit is what reconstructs it. Maximum fit error: 0.0090.
- The effective yield is $\alpha = \bar{\alpha}\,\alpha_{\max}(\mu_e, \sigma_e)$ with $\alpha_{\max}$ the conservative energy bound of paper Eq. 33.

5.3 The U channel and the two albedo methods

The fourth channel U is a UV Gaussian CMF. The albedo's U coordinate, ρ_U, is needed to build $\bar{\mathbf{R}}$ but is not available from an RGB colour. It is computed in two ways:

$$\text{Method 1 (spectral):} \quad \rho_U = \mathbf{r}^\mathsf{T}\,\mathbf{S}_{:,U}, \tag{15}$$

$$\text{Method 2 (projection):} \quad \rho_U = \text{clip}(\mathbf{t}_U^\mathsf{T}\boldsymbol{\rho}_{XYZ},\, 0, 1), \qquad \mathbf{t}_U = \mathbf{S}_{:,U}^\mathsf{T}\,\tilde{\mathbf{S}}_3, \tag{16}$$

where $\tilde{\mathbf{S}}_3$ is the dual of the XYZ sub-basis. Method 1 needs the spectrum and is the reference; Method 2 is a 3-vector dot product (paper Eq. 31) and is what a shader can do; it ships as `UV_WEIGHTS`. Every figure is produced for both, and `u_channel_comparison.png` (Figure 5) plots the two ρ_U across all materials.

5.4 Six pipelines

This table gives a summary of our experiment. The reference is reduced *after* the last bounce; every model is reduced *before* the first.

Pipeline	Operator	Light enters as	Dim. during transport	Reduced
Spectral ref.	$\mathbf{P}$ (Eq. 13)	$\mathbf{L}_0$	$N = 500$	after bounce n
BF-reduced	$\bar{\mathbf{P}}_{\text{bf}} = \bar{\mathbf{R}} + \bar{\mathbf{F}}_{\text{bf}}(\mathbf{I} - \bar{\mathbf{R}})$	$\mathbf{c}_0 = \mathbf{S}^\mathsf{T}\mathbf{L}_0$	$K = 4$	before
Analytic-red.	$\bar{\mathbf{P}}_{\text{an}} = \bar{\mathbf{R}} + \bar{\mathbf{F}}_{\text{an}}(\mathbf{I} - \bar{\mathbf{R}})$	$\mathbf{c}_0$	4	before
Hybrid	$\mathbf{H} = \mathbf{D} + \bar{\mathbf{F}}_{\text{an}}(\mathbf{I} - \mathbf{D})$, then $\bar{\mathbf{P}}_{\text{an}}$	$\mathbf{c}_0$	4	before
Naive	$\boldsymbol{\rho}^{\circ n} \circ \mathbf{c}_0$	$\mathbf{c}_0$	4	before
R-only ref.	diag($\mathbf{r}$) (no $\mathbf{F}$)	$\mathbf{L}_0$	500	after

Table 2: Bounce n of a model is $\bar{\mathbf{P}}^{\,n}\mathbf{c}_0$ (Hybrid: $\bar{\mathbf{P}}_{\text{an}}^{n-1}\mathbf{H}\mathbf{c}_0$). The R-only row is not a model: it is the spectral reference with fluorescence switched off, and its ΔE_{2000} against the full reference measures how much of the signal *is* fluorescence.

The gap between BF-reduced and Analytic-reduced isolates one thing only, the Gaussian-fit error of Eq. (14); the gap between BF-reduced and the spectral reference is the metameric-black error of the reduction itself.

5.5 Display and metric

The U coordinate is dropped (paper Eq. 30), normalised XYZ is converted to standard XYZ and then to linear sRGB with the IEC 61966-2-1 matrix, and the swatches are gamma-encoded and clamped to $[0, 1]$. ΔE_{2000} is computed from the *linear* sRGB values via CIELAB under D65, so the metric is a display-space quantity rather than a distance in the normalised basis.

5.6 Experiment and figures

Eight materials, six illuminants, 20 bounces, both U methods, with $\bar{\alpha} = 0.95$, $\mu_a = 380\,\text{nm}$ (UV absorption, to exercise the U channel), $\mu_e = 520\,\text{nm}$, $\sigma_a = 30$, $\sigma_e = 20$. Per material and method the script writes the main panel (Figure 2) and a diagnostic sheet (Figure 3): chromaticity trajectory, luminance and U over bounces, and the spectra with and without fluorescence with the $\bar{u}$ CMF overlaid. For two representative materials it also sweeps (μ_a, μ_e) (Figure 4) and (σ_a, σ_e), and a separate panel compares D65 with a pure-UV Gaussian illuminant.

6 Diagonal versus reduced transport, the authors' figure

`fig_upscale_diag.py` reproduces a supplementary figure supplied by Laurent Belcour, with the authors' private modules replaced by numpy and matplotlib. Eight XYZ inputs are projected to valid spectra ($\tilde{\mathbf{S}}\mathbf{c}$, scaled to a peak of 0.8, clipped), then transported for ten bounces both diagonally and through $\bar{\mathbf{R}}$; the script plots ΔE_{2000} in xy, in Y and in XYZ against bounce, the chromaticity trajectory on the spectral locus, and the reconstructed spectra (Figure 6). Aligning our pipeline on this script fixed three conventions that the rest of the repository inherits: column-normalised CMFs with no extra $\sum \bar{y}$ factor, no chromatic adaptation, and the naive baseline defined as a component-wise power in XYZ.

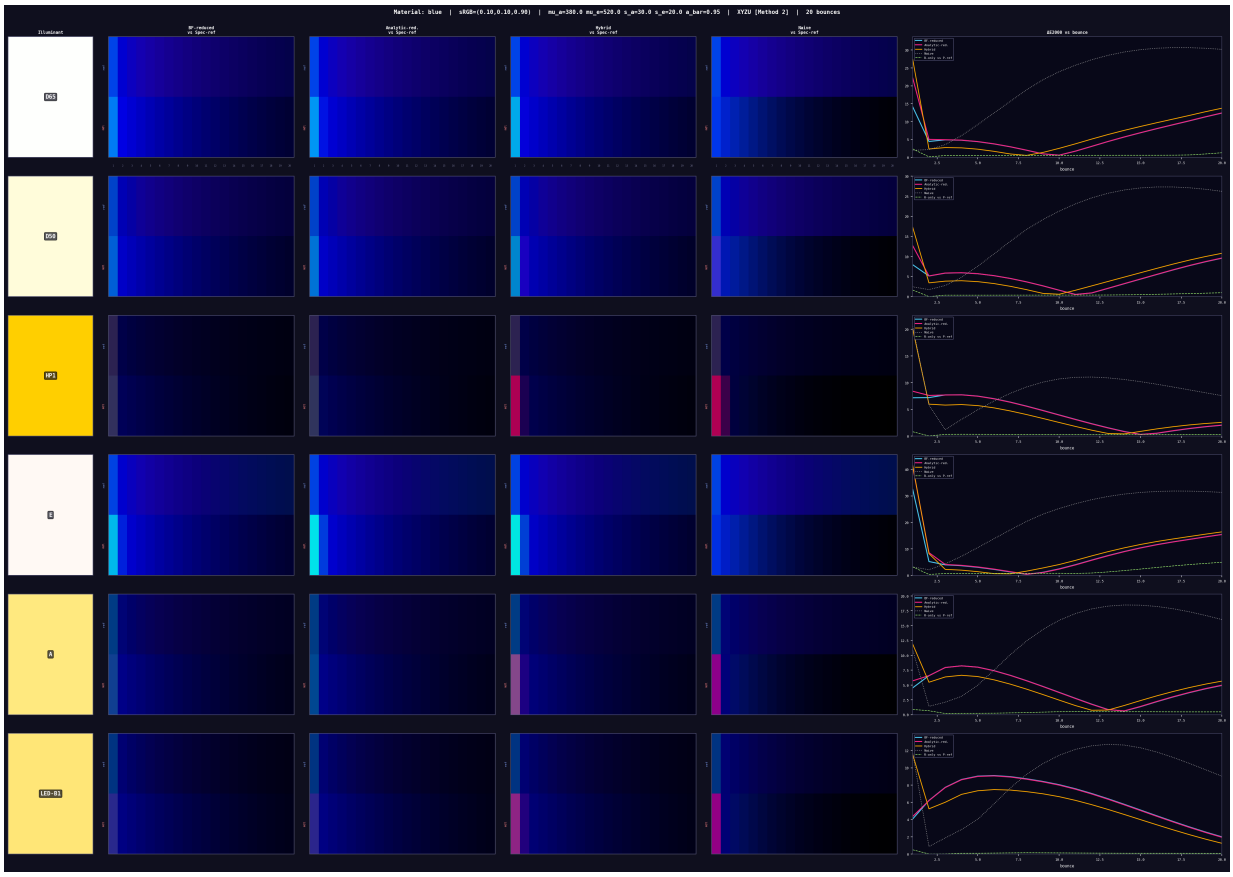

Figure 2: `validate_fluorescence.py`, material *blue*, Method 2. Columns: illuminant, BF-reduced, Analytic-reduced, Hybrid, Naive, ΔE_{2000} vs bounce. Each swatch: reference above, model below, bounces left to right.

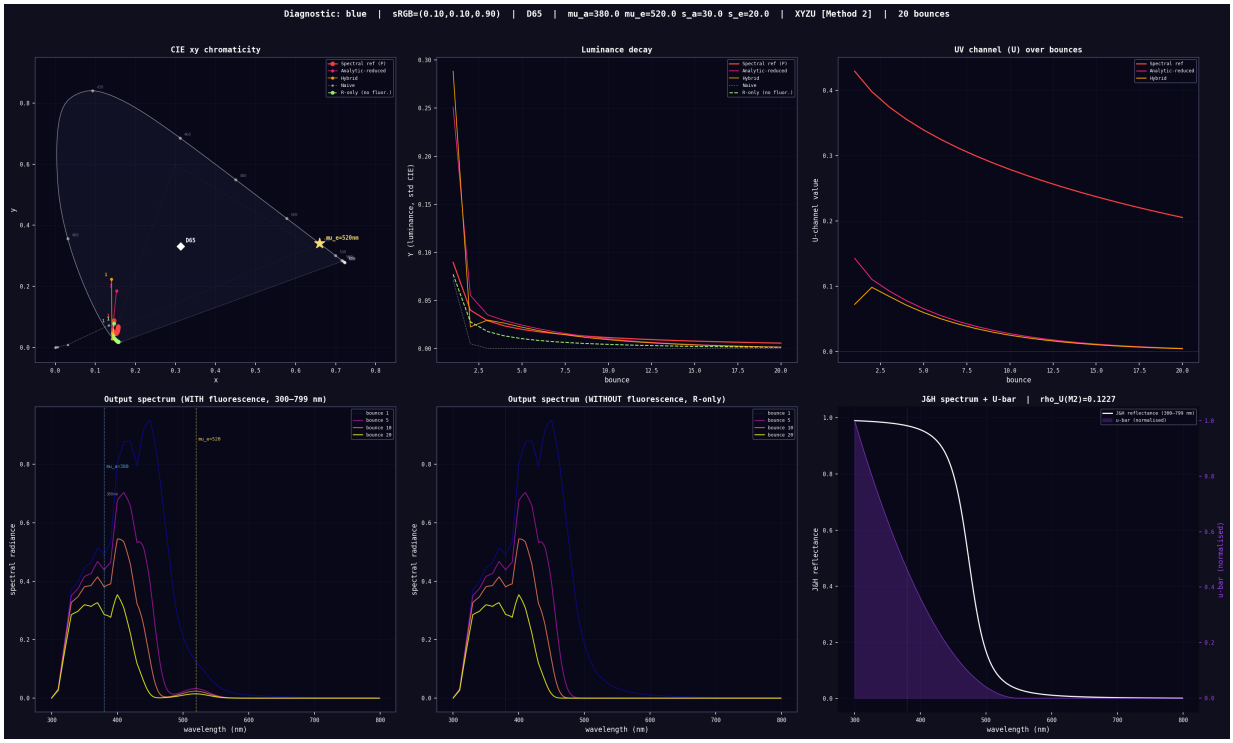

Figure 3: Diagnostics for the same material under D65: CIE xy trajectory, luminance and U against bounce, spectra with and without fluorescence, and the Jakob–Hanika albedo with the $\bar{u}$ function overlaid.

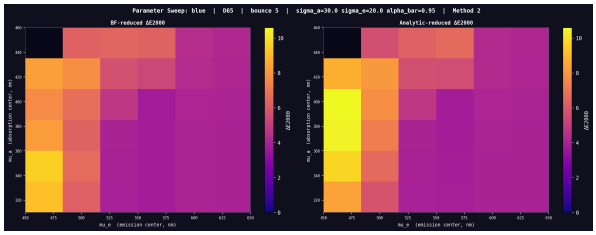

Figure 4: ΔE_{2000} at bounce 5 over the (μ_a, μ_e) plane, material *blue*, Method 2.

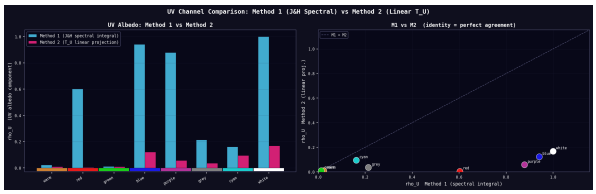

Figure 5: ρ_U from the spectral integral (Method 1) and from the linear projection (Method 2) across the eight materials.

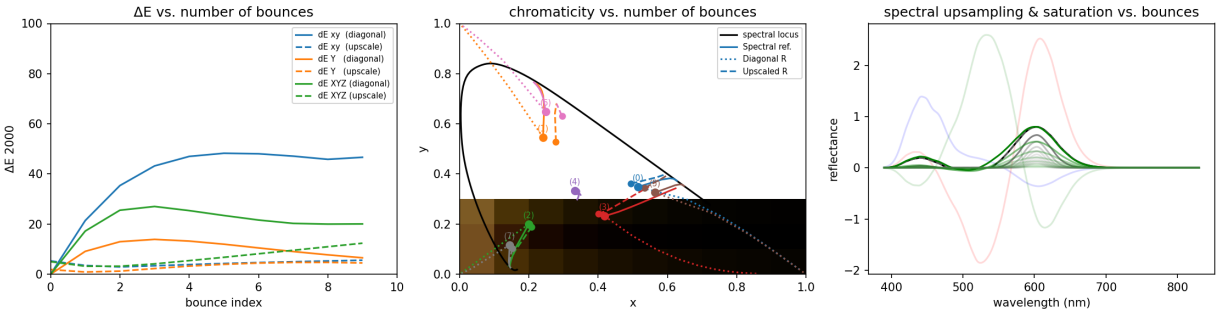

Figure 6: `fig_upscale_diag.py`: ΔE_{2000} vs bounce for diagonal and reduced transport (left), chromaticity drift over bounces (centre), spectral reconstruction and saturation (right).

7 Bridge to the Godot implementation

7.1 Emitted constants and their layout

`generate_godot_matrices.py` rebuilds the XYZU basis exactly as `validate_fluorescence.py` does, derives every fixed matrix, runs the self-checks of Table 4, and prints the literals to paste into the addon. Layout matters because GLSL matrices are column-major while the Python side is row-major:

Constant	Shape	Emitted as	Consumer
`Rx, Ry, Rz, Ru` (Eq. 7)	4×4	GLSL `mat4`, each `vec4` one *column*	shader
`C`	4×4	row-major `float[16]`	shader
`UV_WEIGHTS` $= \mathbf{t}_U$	3	`vec3`	shader
`LIN_SRGB_TO_NORM_XYZ`, inverse	3×3	row-major `float[9]`	shader
`TG_T` (4×5), `TG` (5×4), `C_MAT`	–	GDScript `Array[float]`, row-major	controller

Table 3: The controller computes $\bar{\mathbf{F}}$ on the CPU from Eq. (14) whenever a Gaussian parameter changes and uploads it as a `mat4`; the shader evaluates $\bar{\mathbf{R}}\mathbf{c} + \bar{\mathbf{F}}\big((\mathbf{I} - \bar{\mathbf{R}})\mathbf{c}\big)$ per fragment.

Self-check	Value
$\max \lvert\mathbf{S}_4^{\mathsf{T}}\tilde{\mathbf{S}}_4 - \mathbf{I}_4\rvert$	3.8×10^{-15}
$\max \lvert\mathbf{S}_3^{\mathsf{T}}\tilde{\mathbf{S}}_3 - \mathbf{I}_3\rvert$	9.7×10^{-16}
Gaussian CMF fit, $\max \lvert\mathbf{S}_4 - \mathbf{G}\mathbf{T}_G\rvert$	0.0090
$\max \lvert\mathbf{C} - \mathbf{C}^{\mathsf{T}}\rvert$	2.8×10^{-13}
$\max \lvert\sum_k \mathbf{R}_k - \mathbf{I}_4\rvert$ (intrinsic reduction error of a flat spectrum)	0.108
$\max \lvert$`LIN_TO_NORM` $\cdot$ `NORM_TO_LIN` $- \mathbf{I}_3\rvert$	1.4×10^{-7}

Table 4: Printed by `generate_godot_matrices.py`. The last line is the rounding of the two published sRGB matrices, not a defect. The $\sum_k \mathbf{R}_k$ line is informational: a flat spectrum is not in the span of $\tilde{\mathbf{S}}_4$, so its reduced reflectance is not exactly the identity.

7.2 What the shader does with the albedo

For the record, and because it differs from the reflectance script's treatment, the direct-lighting shader builds its 4-D albedo as

```
vec3  albedo_linear = texture(albedo, UV).rgb;
vec3  albedo_xyz    = LIN_SRGB_TO_NORM_XYZ * albedo_linear;
float albedo_u      = clamp(dot(albedo_xyz, UV_WEIGHTS), 0.0, 1.0);
vec4  rho           = vec4(albedo_xyz, albedo_u);
```

So Method 2 is what the engine uses: ρ_U comes from the linear projection $\mathbf{t}_U$ of Section 5.3, which is why that method is validated next to the spectral integral rather than only mentioned. Only ρ_U is clamped; the XYZ coordinates are used as they come out of the texture conversion. No alpha scaling is applied: the reconstructed reflectance spectrum implied by a saturated authored colour can therefore exceed unity in the engine, which is exactly the regime Section 3.1 identifies as unphysical. Under direct lighting alone the consequence is bounded, but it is the first thing to revisit if the shader is extended to multiple bounces, where Figure 1 shows the error compounding.

7.3 Verification against the Python reference

The following scripts compare the engine's algebra with the validated pipeline and exit non-zero if a check fails, so they also serve as regression tests.

`verify_fbar_convention.py` builds, for one test material ($\mu_a = 430$, $\sigma_a = 40$, $\mu_e = 560$, $\sigma_e = 40$, $\bar{\alpha} = 0.8$), the brute-force $\bar{\mathbf{F}}_{\mathrm{bf}}$, the validated $\bar{\mathbf{F}}_{\mathrm{an}}$, and the controller's $\bar{\mathbf{F}}$ under two conventions: the one first written, and the corrected one. Comparing the *effective* operator each applies to a colour is what exposed two bugs in the original controller:

1. the $\mathbf{F}^{\circ}$ indices were swapped (absorption paired with the emission Gaussian index and vice versa, against paper Eq. 16);
2. the 4×4 was uploaded as a `Projection` whose columns were the rows of $\bar{\mathbf{F}}$, so the shader applied $\bar{\mathbf{F}}^{\mathsf{T}}$.

`verify_godot_shader.py` takes an arbitrary 4-D albedo and 4-D light and checks that the shader's factored evaluation reproduces $(\bar{\mathbf{R}} + \bar{\mathbf{F}}(\mathbf{I} - \bar{\mathbf{R}}))\mathbf{c}$ for both the reduced and the diagonal reflectance, since the shader exposes both.

Script	Check	Value
`verify_fbar_convention`	corrected controller $\bar{\mathbf{F}}$ vs $\bar{\mathbf{F}}_{\mathrm{an}}$	8.0×10^{-17}
	original effective operator vs $\bar{\mathbf{F}}_{\mathrm{an}}$	0.53
	$\lVert\bar{\mathbf{F}}_{\mathrm{an}} - \bar{\mathbf{F}}_{\mathrm{bf}}\rVert_F$ (model approximation)	0.168
`verify_godot_shader`	controller $\bar{\mathbf{F}}$ vs validated	7.0×10^{-17}
	shader output, reduced $\bar{\mathbf{R}}$	1.2×10^{-16}
	shader output, diagonal $\mathbf{D}$	1.1×10^{-16}

Table 5: Current values (`make check`). Everything that should be zero is at double precision; the 0.53 line documents that the pre-fix convention was wrong by a large margin, not by a rounding artefact. The 0.168 line is the closed-form Gaussian model's approximation of the true reradiation matrix for this material; the ΔE_{2000} curves of Section 5 measure its perceptual effect, and it is guarded here only against regression.

8 Consistency notes and caveats

The following points are choices, not bugs, but they affect how the numbers should be read.

1. **Two different bases** (Table 1). The reflectance and fluorescence scripts do not share a grid, a CMF set or a value of K. Albedo coordinates and ΔE_{2000} values are not comparable between them.

2. **Two different ΔE_{2000} clipping conventions.** `validate_fluorescence.py` clips linear sRGB at 0 only before computing ΔE_{2000}; `validate_reflectance.py` clips to $[0, 1]$. In the latter, once both reference and model exceed the display range they both clip to white and the reported ΔE_{2000} collapses, so the curves are optimistic in the saturated regime (visible in Figure 1, bottom). The swatches, clamped in both scripts, do not show the difference.

3. **Two XYZ conventions in the reflectance script.** `validate_reflectance.py` reconstructs its spectrum from standard CIE XYZ but projects and reduces with the column-normalised basis. The round trip is exact (it is checked), so the script is self-consistent, but its $\boldsymbol{\rho}$ is on a different scale from the normalised-XYZ convention of the fluorescence script.

4. **Jakob–Hanika below 380 nm** is an extrapolation of a fit made on the visible range. The default $\mu_a = 380\,\mathrm{nm}$ sits on that boundary. Results in the UV are therefore "plausible", not "measured".

5. **$\mathbf{T}_G$ by dense least squares** rather than the paper's 0/1 grouping, for the normalisation reason given in Section 5.

6. **One constant corrected on conversion.** The notebook from which `validate_reflectance.py` was extracted had the XYZ→sRGB entry $(3, 1)$ as 0.0555434; it is 0.0556434 (IEC 61966-2-1) and was corrected. The effect is below display precision.

7. **Alpha scaling is validated but not ported.** The engine applies neither upsampling variant to authored albedos (Section 7.2); it consumes the albedo texture directly. This is a known gap, not an oversight in the validation.

8. **Analytic-model error is material-dependent.** The 0.168 of Table 5 is for one test material; the parameter sweeps (Figure 4) map how it varies with the absorption and emission positions.

9 Reproduction

```
make venv            # python3 -m venv .venv ; pip install -r requirements.txt
make check           # verify_fbar_convention + verify_godot_shader, exit 1 on failure
make reflectance     # results/reflectance/        (~1 min)
make fluorescence    # results/fluorescence/       (several minutes)
make upscale         # results/upscale_diag/       (~10 s)
make matrices        # print the GLSL / GDScript constants
make doc             # this document (latexmk), after ‘make figures‘
```

All scripts resolve `data/` and `results/` relative to their own location (`scripts/_paths.py`) and can be run from any directory. Pinned versions: Python 3.12, numpy 2.4.6, scipy 1.17.1, matplotlib 3.10.9, colour-science 0.4.7. Every figure in `results/` is committed and was produced with these versions.

A File map

Path	Role
`scripts/validate_reflectance.py`	Section 4; alpha scaling, Hybrid, smoothed illuminant
`scripts/validate_fluorescence.py`	Section 5; six pipelines, two ρ_U methods, sweeps
`scripts/fig_upscale_diag.py`	Section 6; authors' figure, standalone
`scripts/generate_godot_matrices.py`	Section 7; emits engine constants with self-checks
`scripts/verify_fbar_convention.py`	Section 7; controller $\bar{\mathbf{F}}$ check, documents the two bugs
`scripts/verify_godot_shader.py`	Section 7; end-to-end shader algebra check
`scripts/_paths.py`	repository-relative paths
`data/cmf/xyzu.csv`	XYZU CMFs, 300–799 nm (Fichet et al. 2024 companion)
`data/cmf/ciexyz06_2deg.csv`	CIE 2006 2° XYZ (Belcour et al. 2025 supplement)
`data/srgb.coeff`	Jakob–Hanika sRGB coefficient table
`results/`	committed figures, one folder per script

References

- L. Belcour, A. Fichet, P. Barla. *A Fluorescent Material Model for Non-Spectral Editing & Rendering.* SIGGRAPH 2025. hal-05267431.
- A. Fichet, L. Belcour, P. Barla. *Non-Orthogonal Reduction for Rendering Fluorescent Materials in Non-Spectral Engines.* Computer Graphics Forum, 2024.
- W. Jakob, J. Hanika. *A Low-Dimensional Function Space for Efficient Spectral Upsampling.* Computer Graphics Forum (Eurographics), 2019.